HCMC University Of Technology
Faculty of Computer Science & Engineering

Course: Computer network
Implement stream-based HTTP server and chat application

October 30, 2026

Goal: The objective of this assignment is the application of major components in a computer network, for example, client server paradigm, the peer-to-peer paradigm and the network programming.

Content: In detail, student will practice with three major modules: the client-server HTTP communication, the peer-to-peer based chat application and the streaming-socket connection, which includes

- client processes and server processes.
- multiple peer processes connect together.
- the non-blocking network communication.
- the streaming connection-less transport socket.
- the stream-based HTTP server

Besides, student will practice the design and implementation of Simple peer-to-peer protocol via TCP/IP.

Result: After this assignment, student can understand partly the principle of a computer network system. They can understand and design the role of each types of processes, i.e server process, client process and tracker peer process in a network communication.

Contents

1 Introduction **3**
1.1 An overview . . . 3
1.2 Source Code . . . 3
1.3 Server process . . . 4
1.4 How to Create a Server Process? . . . 4
1.5 How to Run the Server . . . 5

2 Implementation **7**
2.1 Stream-based raw socket . . . 7
2.2 Implement the authentication for HTTP server . . . 8
2.3 Implement hybrid chat application . . . 9
2.4 Put It All Together . . . 11

3 Submission **12**
3.1 Source code . . . 12
3.2 Requirements . . . 12
3.3 Grading . . . 12
3.4 Code of ethics . . . 12

1 Introduction

1.1 An overview

In this work, students implement a HTTP server application equipped with a webapp chat application. The server application has multiple server process deployments with a proxy server process and multiple backend server process. A WeApRous framework deployment will help provide the environment for implementing peer-to-peer application

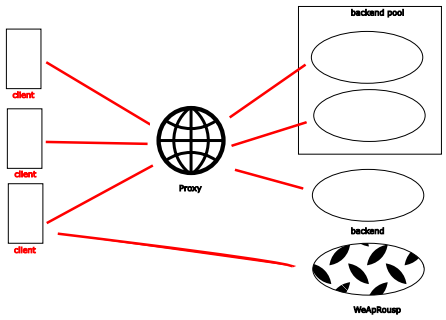


Figure 1: The general view of key modules in this assignment

Through those modules, the server framework allows multi-processes created by users to communicate and use the computing resources. .

1.2 Source Code

After downloading the source code of the assignment in the *Resource* section on the portal platform and extracting it, you will see the source code organized as follows.

- http_daemon
  - `daemon`: response.py, request.py, backend.py, httpadapter.py, weaprous.py, dictionary.py
  - `streaprous_socket`: raw socket support connection-oriented transport layer
  - `modules`: the directory structure at this level is as follow
    - database:
    - cluster
    - dav: Distributed Authoring and Versioning (DAV) functionality
    - http
    - http2
    - ldap: mod_ldap.py
    - proxy: mod_proxy_balancer.py, mod_proxy_express.py, mod_proxy_lbm.py, mod_proxy_tunnel.c
    - session: mod_session_cookie.py, mod_session_crypto.py, session_dbd.py
    - ssl mod_ssl.py
  - `www`: default location for static HTML webpages
    - index.html
  - `static`: default location for other HTML static objects
    - `images`: default location for images

    * `css`: default location for css file
    * `js`: default location for js
  - `apps`: default location for custom webapp
  - `cert`: default location for certification
  - `db`: default location for database
  - `config`: default location for proxy configuration file
  - `start_proxy.py`: the main program to start a proxy process
  - `start_backend.py`: the main program to start a backend process
  - `start_sampleapp.py`: the main program to start a webapp (WeApRous) process

1.3 Server process

Proxy The proxy acts as an intermediary between the clients and the backend or webapp services. In this architecture, it receives incoming requests from multiple clients and forwards them to the appropriate backend resources. This process is crucial for managing traffic, optional for security policies, and enable load balancing/forwarding. By abstracting the backend infrastructure from the clients, the proxy helps ensure scalability and resilience, while also allowing for centralized control over routing and access.

Backend The backend refers to the core services or servers that handle the actual processing of client requests. In the diagram, the backend pool includes multiple backend instances, which may represent a set of processed within a logic layers. These components are responsible for executing and generating responses. The "previous step" proxy process routes the client requests to these backend units based on availability, load, or specific routing rules, ensuring efficient and reliable service delivery.

WebApp - StreApRous The WebApp, labeled as "StreApRous" (short form of StreamAppRoutes) in the image, represents a specialized group of web-facing applications or services that deliver data to clients as a continuous stream. These are typically user-facing components, such as web servers or frontend applications, that send content incrementally rather than waiting for a complete response. StreApRous may be optimized for streaming HTML, serving static assets, or interacting with users through RESTful routes. Like the backend, it receives traffic through the proxy, which helps manage access and performance. StreApRous can also run independently without a proxy.

1.4 How to Create a Server Process?

The content of each process is actually a copy of a program stored on disk. Thus to create a process, we must first generate the program which describes its content. A program is defined by a single file with the following format:

Listing 1: "how to create a proxy

```
# From daemon/proxy.py
def create_proxy(ip, port, routes):
    run_proxy(ip, port, routes)
```

Listing 2: "Create a backend

```
# From daemon/backend.py
def create_backend(ip, port, routes={}):
    run_backend(ip, port, routes)
```

Listing 3: "Create a webapp

```
# From ./start_sampleapp.py
app = StreapRous()

@app.route('/op1', methods=['POST'])
def op1(headers="guest", body="anonymous"):
    ...

@app.route('/op2', methods=['PUT'])
def op2(headers="guest", body="anonymous"):
    ...
```

1.5 How to Run the Server

What we are going to do in this assignment is to implement a simple HTTP server. To start these server process, we must create a script that revokes these server process and provide the hardware and the environment that we will work. The description file is defined in the following format (adapt these IP socket address to your system environment):

```
# From config/proxy.conf
host "192.168.56.103:8080" {
    proxy_pass http://192.168.56.103:9000;
}

host "app1.local" {
    proxy_pass http://192.168.56.103:9001;
}

host "app2.local" {
    proxy_set_header Host $host;

    proxy_pass http://192.168.56.210:9002;
    proxy_pass http://192.168.56.220:9002;

    dist_policy round-robin
}
```

The configuration basic structure Each configuration block begins with a host declaration followed by a set of instructions enclosed in

- host: declares the target hostname or IP address and optional port.
- Block {}: contains directives that define how requests to this host should be handled.

Supported directives

- `proxy_pass [URL];`: defines the backend server to which requests should be forwarded. You can specify multiple `proxy_pass` entries to enable load balancing policy.
- `proxy_set_header [HeaderName] [Value];`: sets custom headers for forwarded requests. This example alternate the original Host header field in the client requests.
- `dist_policy [strategy];` Specifies the distribution policy when multiple backends are defined.

- round-robin: distributes requests evenly across all backends.
- Other strategies (if supported) might include least-conn, random, etc.

Listing 4: "Start the processes"

```
#    Entry point for launching the server. This block parses command-line
#    arguments to determine the server's IP address and port.
#    :arg --server-ip (str): IP address to bind the server (default: 127.0.0.1).
#    :arg --server-port (int): Port number to bind the server (default: 9000).
    parser = argparse.ArgumentParser(prog='Proxy', description='', epilog='Proxy
        daemon')
    parser.add_argument('--server-ip', default='0.0.0.0')
    parser.add_argument('--server-port', type=int, default=PROXY_PORT)

    args = parser.parse_args()
    ip = args.server_ip
    port = args.server_port

# From start_proxy.py
from daemon import create_proxy

routes = parse_virtual_hosts("config/proxy.conf")
create_proxy(ip, port, routes)

# From start_backend.py
from daemon import create_backend

create_backend(ip, port)

# From start_sampleapp.py
from daemon.streaprous import StreapRous

# Prepare and launch the RESTful application
app.prepare_address(ip, port)
app.run()
```

2 Implementation

2.1 Stream-based raw socket

The stream processing following flow shows how two peer daemons establish a raw TCP connection and exchange data as a continuous byte stream.

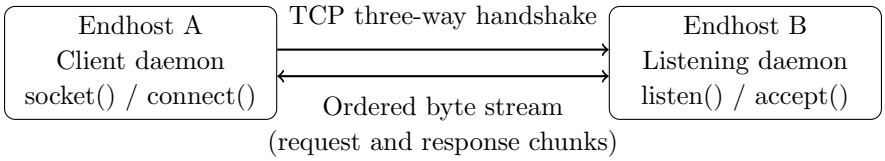


Figure 2: Raw socket connection flow between peer daemons

Transport connection management The connection manager stores a record for each active peer connection. Each `ConnectionInfo` entry tracks the client and server addresses and ports, the connection state, sequence numbers, the expected final acknowledgment, and the time of the last activity. Records are indexed by the client IP address and port so incoming packets can be matched to the correct connection and its state updated as data is exchanged.

Listing 5: Global Connection State Management

```
# Connection structure
# Connection structure
# This structure holds connection and its state information
conn_struct = namedtuple('conn_struct', [
    'client_ip',
    'client_port',
    'server_ip',
    'server_port',
    'state',
    'server_seq',
    'client_seq',
    'expected_final_ack',
    'last_activity'
])


# Global table to manage multiple connection
# The mapping include {key: value} as { (client_ip, client_port): conn_struct }
connections = {}
```

Connection handshake On the listening daemon, `accept()` completes the TCP three-way handshake with a connecting peer and returns a `conn_info` record for the established connection.

Listing 6: Connection accept

```
# Accept connection using raw socket handshake
conn_info = accept(server, ip, port, interface)
```

Connection receive `run_conn_recv()` reads available bytes from the established raw socket identified by `conn_info`. The caller can process the received data as stream chunks; an empty result indicates that no data was received or the connection has closed.

Listing 7: Connection receive

```
# Receive data using streaprous raw connection
recvdata = run_conn_recv(raw_socket, conn_info)

if not recvdata:
    print("[HttpAdapter] No data received or connection closed")
    return
else:
    print(recvdata)
```

Connection send `run_conn_send()` transmits the encoded payload through the established raw socket connection identified by `conn_info`.

Listing 8: Connection send

```
# Send data using streaprous raw connection
payload = "Data".encode("utf-8")
run_conn_send(raw_socket, conn_info, payload)
```

2.2 Implement the authentication for HTTP server

Description To authenticate a user, there are two common approaches include HTTP headers and Cookies. The former RFC 2617 or the updated version RFC 7235 with extended proxy usecase, specifies HTTP use the header "WWW-Authenticate" to request for an authentication; then, the browser responds with an "Authorization" header that carries user login details. The latter RFC 6265, uses cookies to store with "Set-Cookie" header when a user log in successfully. The server sends back an instruction that stores a small piece of data in user browser. From then on, the browser automatically includes that cookie in every request, so the server knows it's still that logged-in user.
Notes it is recommended to use incognito tab since HTTP apply conditional GET by default. The browser cache can be cleared to perform the system experiment properly.
Task requirements Implement the authentication mechanism. Refers the RFCs for the technical details.

- RFC 2617 - HTTP Authentication: Basic and Digest Access Authentication
  - `https://www.rfc-editor.org/rfc/rfc2617`
- RFC 7235 - Hypertext Transfer Protocol (HTTP/1.1): Authentication
  - `https://www.rfc-editor.org/rfc/rfc7235`
- RFC 6265 - HTTP State Management Mechanism (Cookies)
  - `https://www.rfc-editor.org/rfc/rfc6265`

In Figure 3, a proxy is started at port 8080 and a backend is started at port 9000. User browser surf the webpage at `http://IP:8080` and is redirected to be served at backend `http://IP:9000` address

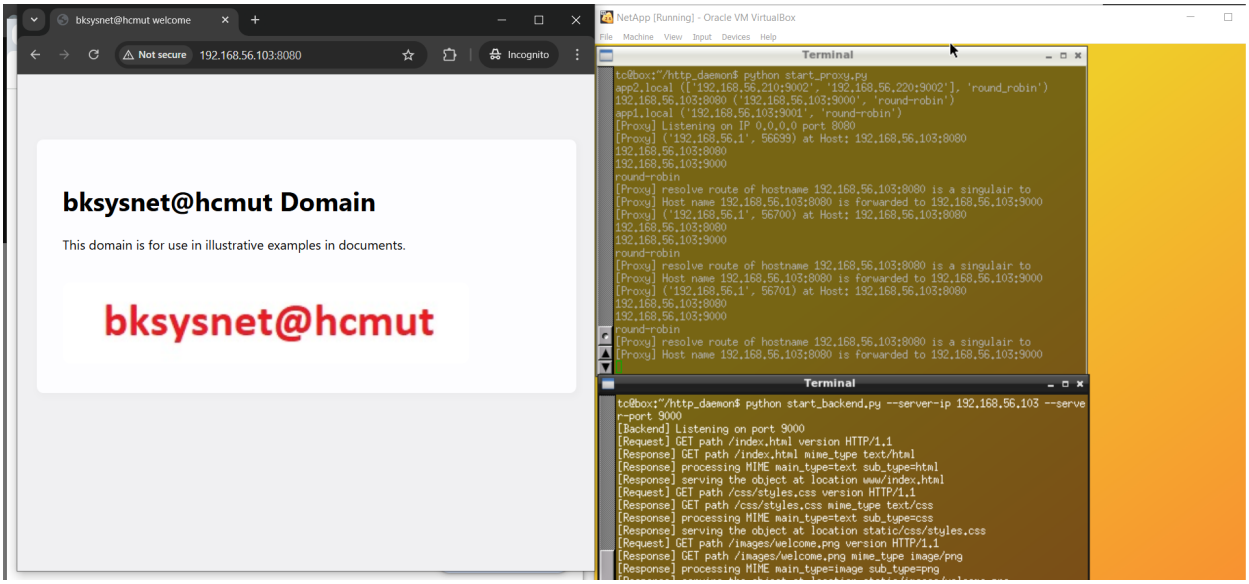


Figure 3: The screenshot of starting a proxy and backends

2.3 Implement hybrid chat application

Description Develop a hybrid network application that has a chat system (similar to skype), combining both client-server and peer-to-peer (P2P) paradigms. The application must support channel management, and synchronization across distributed peers.

Equipped streaming-based connection webapp - StreapRous The webapp support RESTful (Representational State Transfer) is an architectural style for designing networked applications. It relies on standard HTTP methods [GET], [POST], [PUT], [DELETE] to perform operations on resources, which are typically represented as URLs.

RESTful APIs are stateless, scalable, and easy to integrate across platforms. Custom route URLs allow developers to define meaningful and readable endpoints. These routes improve clarity and make your API friendly to handle more complex logic including query parameters or nested paths.

In Figure 4, a user webapp is started at port `http://IP:8000` with two handler at method [PUT] path "/login" and method [POST] path "/hello". The handler is installed by user webapp and is pres

Core functional requirements the application has these following requirements

Initialization phase (Client-Server paradigm)

- *Peer registration:* When a new peer joins, it must submit its IP and port to the centralized server.
- *Tracker update:* The centralized server must maintain a tracking list of active peers.
- *Peer discovery*: Peers can request the current list of active peers from the server.
- *Connection setup:* Peers use the tracking list to initiate direct P2P connections.

Peer chatting phase (Peer-to-Peer Paradigm)

- *Broadcast connection:* A peer must broadcast messages to all connected peers.
- *Direct peer communication:* Peers exchange messages without routing through the centralized server during live sessions.

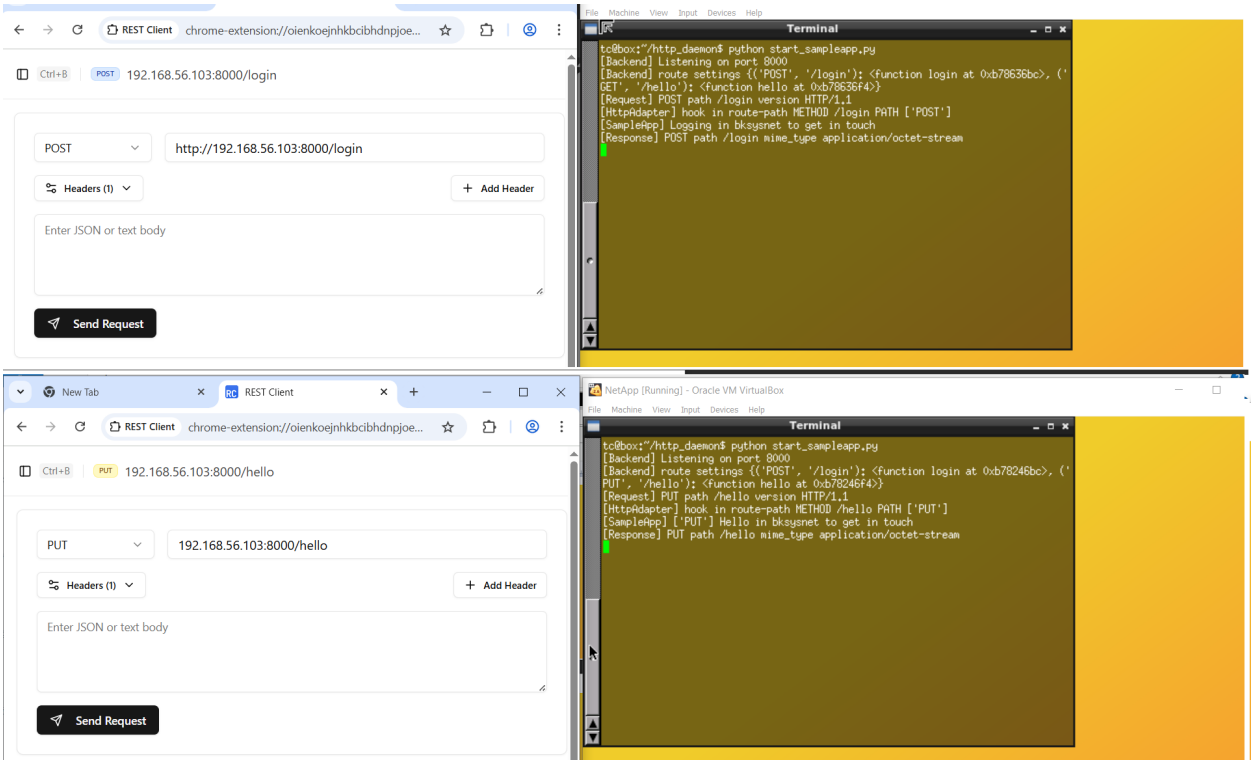


Figure 4: The screenshot of webapp (StreapRous) serves request at `[PUT]/login` and `[POST]/hello`

Channel management

- *Channel listing:* Users can view channels they've joined.
- *Message display:* Each channel has a scrollable message window.
- *Message submission:* UI must support text input and submission.
- *No edit/delete:* Messages are immutable once sent.
- *Notification system:* Users must be notified when new messages arrive.
- *Access control (Optional):* Channels may define custom access policies.

Task requirements

- Streaming-based connection among multi-daemons. Student can based on `streaprous_socket` to implement transport (socket) connection from peer daemon to other peers. The provided streaming (raw socket) supports HTTP request/response.
- Non-blocking communication are required among multi-daemons. Student can based on callback or coroutine for each daemon in each peer to communicate with other peers. Javascript is only allowed in client-side to perform user update or GUI interaction asynchronously. The backend must be implemented using the standard Python library, and the use of JavaScript or existing web framework is strictly forbidden.
- Client-server process programming
- Protocol design in message communication and the processing procedure

  - **login API** example `http://IP:port/login/`
  - **submit-info API** example `http://IP:port/submit-info/`
  - **add-list API** example `http://IP:port/add-list/`
  - **get-list API** example `http://IP:port/get-list/`
  - **connect-peer API** example `http://IP:port/connect-peer/`
  - **broadcast-peer API** example `http://IP:port/broadcast-peer/`
  - **send-peer API** example `http://IP:port/send-peer/`
- Concurrency
- Error Handling

2.4 Put It All Together

Finally, we combine both HTTP Cookies work and StreapRous derivationvs to form a complete server application.

3 Submission

3.1 Source code

Requirement: you have to code the HTTP server and application followed by the coding style. References:

- PEP 8 – Style Guide for Python Code - https://peps.python.org/pep-0008 and
- PEP 257 – Docstring Conventions - https://peps.python.org/pep-0257/

3.2 Requirements

Assignment 1 requirements are folloed the the established Assignment 1 description.

3.3 Grading

The grading of this project is followed the established Assignment 1 description.

3.4 Code of ethics

Faculty staff members involved in code development reserved all the copyright of the project source code.
Source Code License Grant: Author(s) hereby grant(s) to Licensee personal permission to use and modify the Licensed Source Code for the sole purpose of studying while attending the course CO3093/CO3094 at HCMUT.

Revision History

Revision	Date	Author(s)	Description
1.0	10.2025	pdnguyen,	Initialize HTTP server, WeApRous Webapp framework
2.0	03.2026	pdnguyen,	Add non-blocking network communication
2.1	10.2026	pdnguyen,	Add streaming-based network communication

Streaprous documentation

Streaprous is a high-performance HTTP server implementation using raw sockets for external connections and standard sockets for localhost.

Contents:

- daemon Package
  - daemon.backend
  - daemon.httpadapter
  - daemon.request
  - daemon.response
  - daemon.dictionary
  - daemon.proxy
  - daemon.asynaprous
  - daemon.utils
  - start_backend
  - apps
  - apps.sampleapp
  - start_backend
- streprous_socket Package
  - streprous_socket.streaprous_raw_connection
  - streprous_socket.streaprous_packet_dispatcher

Indices and tables

- :ref:`genindex`
- :ref:`modindex`
- :ref:`search`

daemon Package

daemon.backend

daemon.backend

This module provides a backend object to manage and persist backend daemon. It implements a basic backend server using streaprous_raw_connection for raw socket handling. It supports handling multiple client connections concurrently and routing requests using a custom HTTP adapter.

Requirements:

- streaprous_raw_connection: provides raw socket networking interface.
- threading: Enables concurrent client handling via threads.
- response: response utilities.
- httpadapter: the class for handling HTTP requests.
- CaseInsensitiveDict: provides dictionary for managing headers or routes.

Notes:

- The server create daemon threads for client handling.
- The current implementation error handling is minimal, socket errors are printed to the console.
- The actual request processing is delegated to the HttpAdapter class.

Usage Example:

```
>>> create_backend("127.0.0.1", 9000, routes={})
```

daemon.backend.handle_client(*ip, port, conn, addr, routes*) [source]

Initializes an HttpAdapter instance and delegates the client handling logic to it.

Parameters:
- **(str)** (*ip*) – IP address of the server.
- **(int)** (*port*) – Port number the server is listening on.
- **(socket.socket)** (*conn*) – Client connection socket.
- **(tuple)** (*addr*) – client address (IP, port).
- **(dict)** (*routes*) – Dictionary of route handlers.

daemon.backend.handle_client_callback(*server, ip, port, conn, addr, routes*) [source]

Initialize connection instance and delegates the client handling logic to it.

Parameters:
- **(str)** (*ip*) – IP address of the server.
- **(int)** (*port*) – Port number the server is listening on.
- **(dict)** (*routes*) – Dictionary of route handlers.

daemon.backend.handle_client_raw(*server, conn_info, routes*) [source]

Handle client connection using streaprous raw connection.

Parameters:
- **server** – Raw socket
- **conn_info** – ConnectionInfo tuple for the established connection
- **(dict)** (*routes*) – Dictionary of route handlers.

async **daemon.backend.handle_client_coroutine**(*reader, writer*) [source]

Coroutine in async communication to initialize connection instance then delegates the client handling logic to it.

Parameters:
- **(StreamReader)** (*reader*) – Stream reader wrapper.
- **write)** (*write (Stream*) – Stream write wrapper.

async **daemon.backend.async_server**(*ip='0.0.0.0', port=7000, routes={}*) [source]

daemon.backend.run_backend(*ip, port, routes, interface=None*) [source]

Starts the backend server, binds to the specified IP and port, and listens for incoming connections using streaprous_raw_connection for external IPs or standard sockets for localhost. Each connection is handled in a separate thread.

Parameters:
- **(str)** (*ip*) – IP address to bind the server.
- **(int)** (*port*) – Port number to listen on.
- **(dict)** (*routes*) – Dictionary of route handlers.
- **optional)** (*interface (str,*) – Network interface to bind to. Defaults to platform default.

daemon.backend.run_backend_standard(*ip, port, routes*) [source]

Starts the backend server using standard sockets for localhost connections.

Parameters:
- **(str)** (*ip*) – IP address to bind the server.
- **(int)** (*port*) – Port number to listen on.
- **(dict)** (*routes*) – Dictionary of route handlers.

daemon.backend.create_backend(*ip, port, routes={}, interface=None*) [source]

Entry point for creating and running the backend server.

Parameters:
- **(str)** (*ip*) – IP address to bind the server.
- **(int)** (*port*) – Port number to listen on.
- **optional)** (*interface (str,*) – Dictionary of route handlers. Defaults to empty dict.
- **optional)** – Network interface to bind to. Defaults to platform default.

daemon.httpadapter

daemon.httpadapter

This module provides a http adapter object to manage and persist http settings (headers, bodies). The adapter supports both raw URL paths and RESTful route definitions, and integrates with Request and Response objects to handle client-server communication.

class **daemon.httpadapter.HttpAdapter**(*ip*, *port*, *conn*, *connaddr*, *routes*) [source]

Bases: `object`

A mutable `HTTP adapter` for managing client connections and routing requests.

The *HttpAdapter* class encapsulates the logic for receiving HTTP requests, dispatching them to appropriate route handlers, and constructing responses. It supports RESTful routing via hooks and integrates with `Request` and `Response` objects for full request lifecycle management.

ip

IP address of the client.

Type: str

port

Port number of the client.

Type: int

conn

Active socket connection.

Type: socket

connaddr

Address of the connected client.

Type: tuple

routes

Mapping of route paths to handler functions.

Type: dict

request

Request object for parsing incoming data.

Type: Request

response

Response object for building and sending replies.

Type: Response

`conn_info`

streaprous connection info for raw socket.

Type: ConnectionInfo

`raw_socket`

Raw socket for streaprous communication.

Type: socket

`__init__`(*ip*, *port*, *conn*, *connaddr*, *routes*) [source]

Initialize a new HttpAdapter instance.

Parameters:

- **(str)** (*ip*) – IP address of the client.
- **(int)** (*port*) – Port number of the client.
- **(socket)** (*conn*) – Active socket connection.
- **(tuple)** (*connaddr*) – Address of the connected client.
- **(dict)** (*routes*) – Mapping of route paths to handler functions.

`ip`

IP address.

`port`

Port.

`conn`

Connection

`connaddr`

Conndection address

`routes`

Routes

`request`

Request

`response`

Response

`conn_info`

streaprous connection info

`raw_socket`

raw socket for streaprous communication

handle_client(*conn*, *addr*, *routes*) [source]

Handle an incoming client connection.

This method reads the request from the socket, prepares the request object, invokes the appropriate route handler if available, builds the response, and sends it back to the client.

Parameters:
- **(socket)** (*conn*) – The client socket connection.
- **(tuple)** (*addr*) – The client's address.
- **(dict)** (*routes*) – The route mapping for dispatching requests.

handle_client_raw(*raw_socket*, *conn_info*, *routes*) [source]

Handle an incoming client connection using streaprous raw connection.

This method reads the request from the raw socket, prepares the request object, invokes the appropriate route handler if available, builds the response, and sends it back to the client using raw sockets.

Parameters:
- **(socket)** (*raw_socket*) – The raw socket for communication.
- **(ConnectionInfo)** (*conn_info*) – The connection info for the client.
- **(dict)** (*routes*) – The route mapping for dispatching requests.

async **handle_client_coroutine**(*reader*, *writer*) [source]

Handle an incoming client connection using stream reader writer asynchronously.

This method reads the request from the socket, prepares the request object, invokes the appropriate route handler if available, builds the response, and sends it back to the client.

Parameters:
- **(socket)** (*conn*) – The client socket connection.
- **(tuple)** (*addr*) – The client's address.
- **(dict)** (*routes*) – The route mapping for dispatching requests.

property **extract_cookies**

Build cookies from the `Request` headers.

:param req:(Request) The `Request` object. :type resp: :param resp: (Response) The res:class:*Response <Response>* object. :rtype: cookies - A dictionary of cookie key-value pairs.

build_response(*req*, *resp*) [source]

Builds a `Response` object

Parameters:
- **req** – The `Request` used to generate the response.
- **resp** – The response object.

Return type: Response

build_json_response(*req*, *resp*) [source]

Builds a `Response` object from JSON data

Parameters:
- **req** – The `Request` used to generate the response.
- **resp** – The response object.

Return type: Response

`add_headers`(*request*) [source]

Add headers to the request.

This method is intended to be overridden by subclasses to inject custom headers. It does nothing by default.

Parameters: **request** – `Request` to add headers to.

`build_proxy_headers`(*proxy*) [source]

Returns a dictionary of the headers to add to any request sent through a proxy.

`HttpAdapter`.

Parameters: **proxy** – The url of the proxy being used for this request.
Return type: dict

daemon.request

daemon.request

This module provides a Request object to manage and persist request settings (cookies, auth, proxies).

class `daemon.request.Request` [source]

Bases: `object`

The fully mutable "class" *Request <Request>* object, containing the exact bytes that will be sent to the server.

Instances are generated from a "class" *Request <Request>* object, and should not be instantiated manually; doing so may produce undesirable effects.

Usage:

```
>>> import deamon.request
>>> req = request.Request()
## Incoming message obtain aka. incoming_msg
>>> r = req.prepare(incoming_msg)
>>> r
<Request>
```

`__init__`() [source]

`method`

HTTP verb to send to the server.

`url`

HTTP URL to send the request to.

`headers`

dictionary of HTTP headers.

`path`

HTTP path

`body`

request body to send to the server.

`routes`

Routes

`hook`

Hook point for routed mapped-path

`extract_request_line`(*request*) [source]

`prepare_headers`(*request*) [source]

Prepares the given HTTP headers.

`fetch_headers_body`(*request*) [source]

Prepares the given HTTP headers.

`prepare`(*request*, *routes=None*) [source]

Prepares the entire request with the given parameters.

`prepare_body`(*data*, *files*, *json=None*) [source]

`prepare_content_length`(*body*) [source]

`prepare_auth`(*auth*, *url=''*) [source]

`prepare_cookies`(*cookies*) [source]

daemon.response

daemon.response

This module provides a :class: *Response* <*Response*> object to manage and persist response settings (cookies, auth, proxies), and to construct HTTP responses based on incoming requests.

The current version supports MIME type detection, content loading and header formatting

class **daemon.response.Response**(*request=None*) [source]

Bases: `object`

The `Response` object, which contains a server's response to an HTTP request.

Instances are generated from a `Request` object, and should not be instantiated manually; doing so may produce undesirable effects.

`Response` object encapsulates headers, content, status code, cookies, and metadata related to the request-response cycle. It is used to construct and serve HTTP responses in a custom web server.

Attrs status_code (int):	HTTP status code (e.g., 200, 404).
Attrs headers (dict):	dictionary of response headers.
Attrs url (str):	url of the response.
Attrsencoding (str):	encoding used for decoding response content.
Attrs history (list):	list of previous Response objects (for redirects).
Attrs reason (str):	textual reason for the status code (e.g., "OK", "Not Found").
Attrs cookies (CaseInsensitiveDict):	response cookies.
Attrs elapsed (datetime.timedelta):	time taken to complete the request.
Attrs request (PreparedRequest):	the original request object.

Usage:

```
>>> import Response
>>> resp = Response()
>>> resp.build_response(req)
>>> resp
<Response>
```

__init__(*request=None*) [source]

Initializes a new `Response` object.

: params request : The originating request object.

status_code

Integer Code of responded HTTP Status, e.g. 404 or 200.

headers

Case-insensitive Dictionary of Response Headers. For example, `headers['content-type']` will return the value of a `'Content-Type'` response header.

url

URL location of Response.

encoding

Encoding to decode with when accessing response text.

history

A list of `Response` objects from the history of the Request.

reason

Textual reason of responded HTTP Status, e.g. “Not Found” or “OK”.

cookies

A of Cookies the response headers.

elapsed

The amount of time elapsed between sending the request

request

The `PreparedRequest` object to which this is a response.

get_mime_type(*path*) [source]

Determines the MIME type of a file based on its path.

“params path (str): Path to the file.

Rtype str: MIME type string (e.g., ‘text/html’, ‘image/png’).

prepare_content_type(*mime_type='text/html'*) [source]

Prepares the Content-Type header and determines the base directory for serving the file based on its MIME type.

Params mime_type (str): MIME type of the requested resource.
Rtype str: Base directory path for locating the resource.
Raises: **ValueError** – If the MIME type is unsupported.

build_content(*path*, *base_dir*) [source]

Loads the objects file from storage space.

Params path (str): relative path to the file.
Params base_dir (str): base directory where the file is located.
Rtype tuple: (int, bytes) representing content length and content data.

build_response_header(*request*) [source]

Constructs the HTTP response headers based on the class:`Request <Request> and internal attributes.

:params request (class:*Request <Request>*): incoming request object.

Rtypes bytes: encoded HTTP response header.

build_notfound() [source]

Constructs a standard 404 Not Found HTTP response.

Rtype bytes: Encoded 404 response.

build_response(*request*, *envelop_content=None*) [source]

Builds a full HTTP response including headers and content based on the request.

:params request (class:*Request <Request>*): incoming request object.

Rtype bytes: complete HTTP response using prepared headers and content.

daemon.dictionary

class **daemon.dictionary.CaseInsensitiveDict**(**args*, ***kwargs*) [source]

Bases: `MutableMapping`

The `CaseInsensitiveDict` object, which contains a custom behavior of MutuableMapping.

Usage:

```
>>> import tools
>>> word = CaseInsensitiveDict(status_code='404', msg="Not found")
>>> code = word['status_code']
>>> code
404

>>> msg = word['msg']
>>> s.send(r)
Not found

>>> print(word)
{'status_code': '404', 'msg': 'Not found'}
```

__init__(**args*, ***kwargs*) [source]

daemon.proxy

daemon.proxy

This module implements a simple proxy server using Python's socket and threading libraries. It routes incoming HTTP requests to backend services based on hostname mappings and returns the corresponding responses to clients.

Requirement:

- socket: provides socket networking interface.
- threading: enables concurrent client handling via threads.
- response: customized :class: *Response <Response>* utilities.
- httpadapter: :class: *HttpAdapter <HttpAdapter >* adapter for HTTP request processing.
- dictionary: :class: *CaseInsensitiveDict <CaseInsensitiveDict>* for managing headers and cookies.

daemon.proxy.PROXY_PASS
= {'192.168.56.103:8080': ('192.168.56.103', 9000), 'app1.local': ('192.168.56.103', 9001), 'app2.local': ('192.168.56.103', 9002)}

A dictionary mapping hostnames to backend IP and port tuples. Used to determine routing targets for incoming requests.

`daemon.proxy.forward_request`(*host, port, request*) [source]

Forwards an HTTP request to a backend server and retrieves the response.

Params host (str):	IP address of the backend server.
Params port (int):	port number of the backend server.
Params request (str):	incoming HTTP request.
Rtype bytes:	Raw HTTP response from the backend server. If the connection fails, returns a 404 Not Found response.

`daemon.proxy.resolve_routing_policy`(*hostname, routes*) [source]

Handles an routing policy to return the matching proxy_pass. It determines the target backend to forward the request to.

Params host (str):	IP address of the request target server.
Params port (int):	port number of the request target server.
Params routes (dict):	dictionary mapping hostnames and location.

`daemon.proxy.handle_client`(*ip, port, conn, addr, routes*) [source]

Handles an individual client connection by parsing the request, determining the target backend, and forwarding the request.

The handler extracts the Host header from the request to matches the hostname against known routes. In the matching condition,it forwards the request to the appropriate backend.

The handler sends the backend response back to the client or returns 404 if the hostname is unreachable or is not recognized.

Params ip (str):	IP address of the proxy server.
Params port (int):	port number of the proxy server.
Params conn (socket.socket):	client connection socket.
Params addr (tuple):	client address (IP, port).
Params routes (dict):	dictionary mapping hostnames and location.

`daemon.proxy.run_proxy`(*ip, port, routes*) [source]

Starts the proxy server and listens for incoming connections.

The process dinds the proxy server to the specified IP and port. In each incomping connection, it accepts the connections and spawns a new thread for each client using *handle_client*.

Params ip (str):	IP address to bind the proxy server.
Params port (int):	port number to listen on.
Params routes (dict):	dictionary mapping hostnames and location.

`daemon.proxy.create_proxy`(*ip, port, routes*) [source]

Entry point for launching the proxy server.

Params ip (str):	IP address to bind the proxy server.

Params port (int): port number to listen on.
Params routes (dict): dictionary mapping hostnames and location.

daemon.asynaprous

deamon.asynaprous

This module provides a AsynapRous object to deploy RESTful url web app with routing

class **daemon.asynaprous.AsynapRous** [source]

Bases: `object`

The fully mutable `AsynapRous` object, which is a lightweight, mutable web application router for deploying RESTful URL endpoints.

The *AsynapRous* class provides a decorator-based routing system for building simple RESTful web applications. The class allows developers to register route handlers using decorators and launch a TCP-based backend server to serve RESTful requests. Each route is mapped to a handler function based on HTTP method and path. It mappings supports tracking the combined HTTP methods and path route mappings internally.

Usage::

```
>>> import daemon.asynaprous
>>> app = AsynapRous()
>>> @app.route('/login', methods=['POST'])
>>> def login(headers="guest", body="anonymous"):
>>>     return {'message': 'Logged in'}
```

```
>>> @app.route('/hello', methods=['GET'])
>>> def hello(headers, body):
>>>     return {'message': 'Hello, world!'}
```

```
>>> app.run()
```

__init__() [source]

Initialize a new AsynapRous instance.

Sets up an empty route registry and prepares placeholders for IP and port.

prepare_address(*ip*, *port*) [source]

Configure the IP address and port for the backend server.

Parameters:
- **(str)** (*port*) – The IP address to bind the server.
- **(str)** – The port number to listen on.

route(*path*, *methods=['GET']*) [source]

Decorator to register a route handler for a specific path and HTTP methods.

Parameters:
- **(str)** (*path*) – The URL path to route.

- **(list)** (*methods*) – A list of HTTP methods (e.g., ['GET', 'POST']) to bind.

Return type: function - A decorator that registers the handler function.

run() [source]

Start the backend server and begin handling requests.

This method launches the TCP server using the configured IP and port, and dispatches incoming requests to the registered route handlers.

Raise: Error if IP or port has not been configured.

daemon.utils

start_backend

start_backend

This module provides a simple entry point for deploying backend server process using the socket framework. It parses command-line arguments to configure the server's IP address and port, and then launches the backend server.

start_backend.detect_interface(*ip*) [source]

Dynamically detect the network interface for a given IP address.

Parameters: **(str)** (*ip*) – IP address to find interface for

Returns: str: Interface name (e.g., 'lo0', 'eth0', 'en0')

start_backend.setup_rst_block(*ip*, *port*) [source]

Block RST packets from the kernel to allow raw socket server to handle connections properly. On macOS use pfctl, on Linux use iptables.

start_backend.parse_arguments() [source]

Process arguments for server IP, port and Interface Usage: server_tcp.py [PORT] [INTERFACE]

server_tcp.py [IP] [PORT] [INTERFACE]

start_backend.main(*ip*, *port*, *interface=None*) [source]

Main entry point for the backend server.

apps

apps.sampleapp

app.sampleapp

apps.sampleapp.create_sampleapp(*ip*, *port*) [source]

start_backend

start_backend

This module provides a simple entry point for deploying backend server process using the socket framework. It parses command-line arguments to configure the server's IP address and port, and then launches the backend server.

start_backend.detect_interface(*ip*) [source]

Dynamically detect the network interface for a given IP address.

Parameters: **(str)** (*ip*) – IP address to find interface for
Returns: str: Interface name (e.g., 'lo0', 'eth0', 'en0')

start_backend.setup_rst_block(*ip*, *port*) [source]

Block RST packets from the kernel to allow raw socket server to handle connections properly. On macOS use pfctl, on Linux use iptables.

start_backend.parse_arguments() [source]

Process arguments for server IP, port and Interface Usage: server_tcp.py [PORT] [INTERFACE]

server_tcp.py [IP] [PORT] [INTERFACE]

start_backend.main(*ip*, *port*, *interface=None*) [source]

Main entry point for the backend server.

streprous_socket Package

streprous_socket.streaprous_raw_connection

Raw Connection Library A library for handling TCP connections with raw sockets. Based on raw_connection_server.py implementation. Provides connection management, send/receive functionality using raw sockets.

class **streprous_socket.streaprous_raw_connection.ConnectionInfo**(*client_ip*, *client_port*, *server_ip*, *server_port*, *state*, *server_seq*, *client_seq*, *expected_final_ack*, *last_activity*)

Bases: `tuple`

client_ip

Alias for field number 0

client_port

Alias for field number 1

client_seq

Alias for field number 6

expected_final_ack

Alias for field number 7

last_activity

Alias for field number 8

server_ip

Alias for field number 2

server_port

Alias for field number 3

server_seq

Alias for field number 5

`state`

Alias for field number 4

streprous_socket.streaprous_raw_connection.register_keep_port_alive(*listen_ip='0.0.0.0'*, *listen_port=22239*, *retry_interval=10*) [source]

Periodically register and maintain TCP port. Retry registration whenever binding/listening fails. Never accept incoming connections.

Parameters:
- **listen_ip** – IP address to bind to (default: "0.0.0.0")
- **listen_port** – Port to bind to (default: 22239)
- **retry_interval** – Seconds to wait before retrying after failure (default: 10)

Returns: The background thread running the port keep-alive

Return type: threading.Thread

streprous_socket.streaprous_raw_connection.checksum(*msg*) [source]

Calculates a precise internet checksum and returns an integer.

streprous_socket.streaprous_raw_connection.build_tcp_packet(*src_ip*, *dest_ip*, *src_port*, *dst_port*, *seq*, *ack_seq*, *syn=0*, *ack=0*, *fin=0*) [source]

Stitches a structurally flawless 20-byte TCP header.

streprous_socket.streaprous_raw_connection.build_ip_header(*src_ip*, *dest_ip*, *protocol*, *payload_len*) [source]

Builds a structurally flawless 20-byte IP header.

streprous_socket.streaprous_raw_connection.send_tcp_data_segment(*s*, *src_ip*, *dest_ip*, *src_port*, *dst_port*, *seq*, *ack_seq*, *payload_bytes*) [source]

Assembles and transmits a single TCP segment containing data payload.

streprous_socket.streaprous_raw_connection.clean_expired_connections(*timeout=60*) [source]

Background thread to clean up stale or timed-out connections.

streprous_socket.streaprous_raw_connection.create_raw_socket(*interface_name='lo0'*) [source]

Create a raw socket for TCP protocol. Requires root privileges.

Parameters:
- **interface_name** (`str`) – Network interface to bind to (e.g., 'lo0' on macOS, 'lo' on Linux)
- **Note** – This parameter is kept for compatibility but not used for binding to allow receiving packets on all interfaces when listening on 0.0.0.0

Return type: `socket`

Returns: Raw socket object

streprous_socket.streaprous_raw_connection.get_interface_ip(*interface_name=None*) [source]

Get the IPv4 address of a network interface.

Parameters: **interface_name** (`Optional` [`str`]) – Network interface name (e.g., 'enp0s8', 'eth0', 'en0') If None, tries to find the default route interface.

Return type: `str`

Returns: IPv4 address string

streprous_socket.streaprous_raw_connection.accept(*s, listen_ip, listen_port, interface_name=None*) [source]

Accept a new connection by performing the 3-way handshake using raw sockets. Returns a ConnectionInfo tuple for the established connection. Handshakes are serialized to ensure only one 3-way handshake is processed at a time.

Parameters:

- **s** (`socket`) – Raw socket
- **listen_ip** (`str`) – Server IP to listen on (e.g., '0.0.0.0')
- **listen_port** (`int`) – Server port to listen on
- **interface_name** (`Optional` [`str`]) – Network interface name for sending packets (optional)

Returns: Established connection information

Return type: ConnectionInfo

streprous_socket.streaprous_raw_connection.run_conn_recv(*s, conn_info*) [source]

Receive data on an established connection using raw sockets. This function handles incoming packets for a specific connection by reading from the packet dispatcher's per-connection queue.

Parameters:

- **s** (`socket`) – Raw socket (used for sending ACKs)
- **conn_info** (`ConnectionInfo`) – ConnectionInfo tuple for the connection

Returns: The received data payload, or empty bytes if connection closed

Return type: bytes

streprous_socket.streaprous_raw_connection.run_conn_send(*s, conn_info, payload*) [source]

Send data on an established connection using raw sockets.

Parameters:

- **s** (`socket`) – Raw socket
- **conn_info** (`ConnectionInfo`) – ConnectionInfo tuple for the connection
- **payload** (`bytes`) – Bytes payload to send

Returns: Number of bytes sent, or -1 on error

Return type: int

streprous_socket.streaprous_raw_connection.close_connection(*s, conn_info*) [source]

Close a specific connection by sending FIN and removing from connection table.

streprous_socket.streaprous_raw_connection.close_all_connections() [source]

Close all connections and clean up.

streprous_socket.streaprous_packet_dispatcher

Streaprous Packet Dispatcher Module A single thread that reads from raw socket and dispatches packets to per-connection queues. This eliminates GIL contention when multiple threads would call recvfrom() on the same socket.

streprous_socket.streaprous_packet_dispatcher.start_packet_dispatcher(*s, listen_port*) [source]

Start the packet dispatcher thread.

streprous_socket.streaprous_packet_dispatcher.stop_packet_dispatcher() [source]

Stop the packet dispatcher thread.

streprous_socket.streaprous_packet_dispatcher.get_packet_queue(*client_key*) [source]

Get or create packet queue for a connection.

streprous_socket.streaprous_packet_dispatcher.remove_packet_queue(*client_key*) [source]

Remove packet queue for a closed connection.